

TDDE70 Deep Learning – Project description

Fine-Tuning CLIP for Remote Sensing Data

1. Project Overview

The goal of this project is to get hands-on experience with fine-tuning a large vision-language model (CLIP) on remote sensing data, i.e. images collected from satellites and aircrafts. Specifically, you will work with the RSICD dataset, which is a remote sensing image captioning dataset with images collected from Google Earth, Baidu Map, MapABC, and Tianditu. Through this project, you will learn how to adapt a pre-trained CLIP model to a specific domain and evaluate its performance in several relevant tasks, such as zero-shot classification and retrieval.

In addition to fully-supervised fine-tuning, this project also encourages exploring semi-supervised fine-tuning, where both labeled and unlabeled data are incorporated into the training process.

2. Prerequisites

To succeed in this project, all project members should have:

- A solid understanding of deep learning concepts.
- Experience with Python and PyTorch.
- Experience with using a GPU-enabled computing environment for training.

3. Data

The dataset for this project is RSICD¹, which can be downloaded from the RSICD GitHub repository (requires Git LFS) or from the project folder in Lisam. You need to download three files from the repository:

RSICD_images	Folder containing all images in the dataset.
txtclasses_rsicd	Text file containing the class names and their corresponding image file names.
dataset_rsicd.json	JSON file describing each image, including captions and split information (train/val/test).

¹ https://github.com/201528014227051/RSICD_optimal

Dataset split

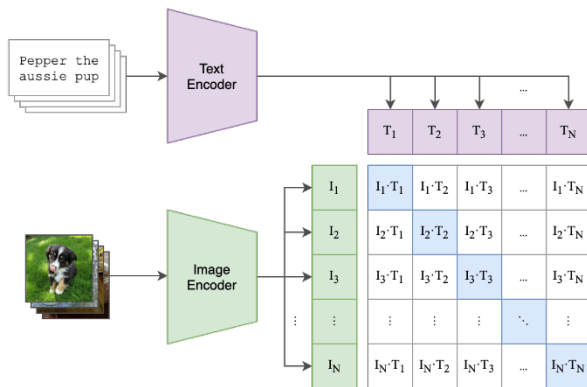
The dataset is originally divided into three subsets for training, validation, and testing, defined in dataset_rscd.json under the "split" tag. However, we will use the available data in a slightly different way in this project, and therefore define the following three data splits:

- **Split A** (referred to as “train” in the json file; 8734 images) - This will be treated as the **unlabeled** dataset for semi-supervised fine-tuning.
Only images, no captions, can be used in this project!
- **Split B** (referred to as “validation” in the json file; 1094 images) - This will be treated as the labeled dataset for training (images with captions). It is suggested to further split this into "real" training and validation sets.
- **Split C** (referred to as “test” in the json file; 1093 images) - This will be treated as the labeled dataset for final evaluation (images with captions and class labels).

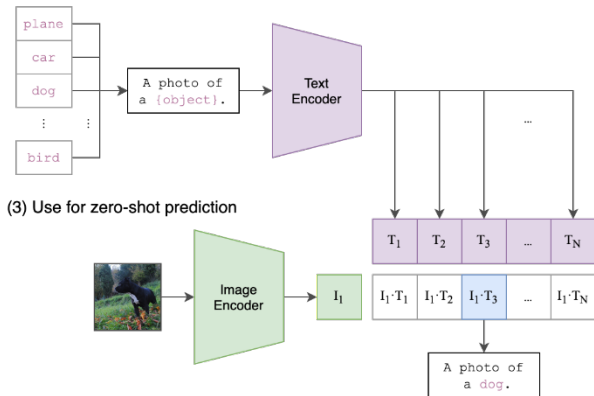
Note that each image is accompanied by five text descriptions (captions). It is recommended to utilize all of the captions from Split B during training.

4. Models

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

Image Credit: <https://github.com/openai/CLIP>

The project will use OpenCLIP², an open-source implementation of CLIP.

Basic model

ViT-B/32: This is the default and basic model for the main experiments.

Optional exploration

If you are interested in exploring other CLIP models you are welcome to do so and can include such results in your report.

² https://github.com/mlfoundations/open_clip

5. Tasks and Experimental Setup

Project settings

This project has two difficulty levels:

Basic version	Fully-supervised fine-tuning using only the labeled dataset with images + captions for training (split B). Performance is tested on the test set (split C).
Advanced version	Semi-supervised fine-tuning using both the labeled dataset (split B) and the unlabeled dataset with only images (split A). You need to design and implement appropriate loss functions to leverage both types of data. Performance is tested on the test set (split C).

Note that you are not allowed to use the test set (split C) for training or tuning of hyperparameters in either of the two versions.

Suggested evaluation tasks on the test set

At a minimum, all students should implement zero-shot classification on the test set. For the advanced version of the project, a few additional tasks of your choice, like image-to-text retrieval, text-to-image retrieval, or other creative extensions related to the project should also be implemented (see *Section 6: Grading Criteria* below).

- **Zero-shot classification [mandatory]** means that a model can classify images into categories without being explicitly trained *as a classifier*. Specifically, the CLIP model can use textual descriptions to assign labels for a given remote sensing image. For example, define text-based descriptions for all categories in this project according to the `txtclasses_rsid` file:

```
"A satellite image of {category A}"
```

```
"A satellite image of {category B}"
```

```
...
```

```
"A satellite image of {category X}"
```

CLIP computes similarity scores between the input image embedding and each text embedding. The category with the highest similarity score is assigned as the predicted label.

- **Zero-shot classification with multiple templates** builds on the idea above but considers many textual descriptions of the same category. That is, instead of just using `"A satellite image of { }"` as a template, you can define new templates and evaluate the similarity scores for all of them. It's up to you to decide how the scores should be combined in a good way.
- **Image-to-text retrieval** aims to retrieve the most relevant textual descriptions for a given image. Note that in this case, the text descriptions from the

dataset_rsicd.json file corresponding to each input image will serve as the ground truth.

- **Text-to-image retrieval** aims to retrieve the most relevant image for a text query. Similar to image-to-text retrieval, the text descriptions from the dataset_rsicd.json file will serve as the text query.
- As mentioned above, feel free to define other relevant tasks and test the model on these!

Instructions for using OpenCLIP base code

You have flexibility in choosing how to utilize OpenCLIP in your project. The official OpenCLIP GitHub repository provides a comprehensive tutorial for easy implementation, which is recommended for a straightforward approach. If you need more flexibility, specifically for the Advanced version of the project, direct modification of the source code is also encouraged to allow for more customization. Note that the main focus for the Advanced version should be on developing novel ideas for semi-supervised learning to effectively fine-tune the CLIP model.

6. Instructions for the final report:

The final report should be a self-contained document formatted according to the **NeurIPS** paper template³ and submitted as a single PDF through Lisam. The main section of the report is limited to **6 pages** (excluding references, ethical considerations, and contribution statements).

The main part of the report must include the following sections:

- **Abstract:** A summary of the project within 150 words. It should concisely explain the problem you aim to solve, your ideas, and how the results reflect your methods.
- **Introduction:** Background and motivation for this study. This is a high-level explanation and motivation of the problem formulation. The introduction should also contain key references to relevant literature. An exhaustive literature review *is not required* for the project, but any related work that you take inspiration from and/or base your solution on should of course be cited and discussed.
- **Problem Formulation:** Formal definition of the problem being addressed, **written in a concise but clear and unambiguous mathematical language**. For instance, if you are solving a classification problem, then you should clarify the goal of the classification in the language of probability theory. Here,

³ <https://media.neurips.cc/Conferences/NeurIPS2024/Styles.zip>

you should clearly state whether you target the basic or the advanced version of the project, and write your problem formulation accordingly.

- **Method:** Description of the approach and techniques used.
- **Results:** Presentation and analysis of experimental results.
- **Conclusion/Discussion:** Summary of insights and potential future work.

In addition to the above, the report should contain the following, not counted towards the 6-page limit:

- An ethical considerations section to reflect on any ethical implications of the work.
- A contribution statement which clearly specifies each group member's individual contributions. This statement should be sufficiently detailed for the examiner to assess each team member individually. **A too vague statement could thus result in a lower grade due to unclarities about how you contributed to the work!**
- Optionally, up to 5 additional pages of appendices for supplementary material (e.g., detailed derivations, extra supporting results), but the main part of the report must be self-contained (i.e., it should be possible to read, understand, and grade the report without the supplementary materials).

7. Grading Criteria

The project will be graded on a U/3/5 scale, with the following criteria:

3 (Pass)	You should achieve the following points. 1. Fully-supervised fine-tuned CLIP and evaluated on zero-shot classification on the test set (basic version of project). 2. Evaluation, analysis, and critical assessment of the results, using good machine learning practices. 3. Clear and solid report as per the instructions provided.
5 (Pass with distinction)	You should achieve the following points. 1. Effective semi-supervised fine-tuning using both labeled and unlabeled data (advanced version of project), along with at least three evaluation tasks. 2. Comprehensive evaluation, analysis, and critical assessment of the results, using good machine learning practices. Some form of ablation studies⁴, parameter analysis⁵, and/or comparisons between different models should be included

⁴ Evaluating the impact of specific components by removing or modifying them.

⁵ Evaluating how different hyperparameters affect performance.

	(but you may not have the time or computational resources to cover all of these aspects). 3. Active participation on the leaderboard (see <i>Section 7: Leaderboard</i>) 4. Clear and solid report as per the instructions provided. We will put particular emphasis on clarity of the mathematical formulation of the problem and chosen approach.
--	---

Furthermore, a passing grade requires active participation in the project seminar, see *Section 9: Project seminar*).

8. Leaderboard

To promote friendly competition and evaluation of your models in a standardized way, we will maintain a leaderboard based on your zero-shot classification performance on a hidden test set. Note that the leaderboard is optional if you only aim for grade 3, but mandatory if you aim for grade 5. **In either case, your grade is completely independent of your performance on the leaderboard.**⁶ However, we still encourage everyone to join the competition, as it can be a fun experience and valuable addition to your portfolio and future applications.

Download the data

We will use a new dataset where the true labels are not accessible to you. Download the dataset (data4leaderboard.zip) from the project folder in Lisam. Alternatively, you can also download it in the leaderboard platform (see the CodaBench instructions). Unzip the file to get a folder of 2,100 images, each of size 256×256×3 pixels.

Target classes

There are 21 land use categories in total:

agricultural, airplane, baseballdiamond, beach, buildings,
chaparral, denseresidential, forest, freeway, golfcourse,
harbor, intersection, mediumresidential, mobilehomepark, overpass,
parkinglot, river, runway, sparseresidential, storagetanks, tenniscourt

You should use these class names as your label space.

⁶ The reason for why we have made it mandatory to participate in the leaderboard for the higher grade is that we think it will be fun to see how the innovative solutions that you come up with in the advanced version of the project will fare!

Prepare your submission

Based on the input images, your model should generate corresponding label predictions. You should enclose all the label predictions in a predictions.txt file using the following format:

```
image_filename predicted_class
image_filename predicted_class
...
```

Please ensure one line per image. `image_filename` must match the file name exactly, including the file extension (e.g., `ak29dkfj8s.jpg`). `predicted_class` must be one of the 21 valid class names listed above. Ensure that every image in the test set is included — missing even a single entry will result in 0 accuracy. Also ensure that the list has no repetition.

As a side note, the ordering of the entries in the list does not matter.

Example predictions.txt:

```
a8s7f1k2jd.jpg river
1kd02ks9dj.jpg airplane
...
```

How to use the CodaBench leaderboard

First, go to https://www.codabench.org/competitions/7051/?secret_key=275c4904-2b2d-4972-99a9-a0098675ff0d. Navigate to “My Submissions” and register for the competition.

N.B. Participation on the leaderboard according to the instruction above requires an account on CodaBench. If you prefer not to register on this third-party platform but still want to participate in the competition (e.g. for fulfilling the requirement for the higher grade), you may upload your predictions.txt file using the submission function in Lisam instead. This will however limit your participation to a single submission (we will submit your solution to CodaBench for you after the deadline) and you will thus not get continuous feedback on how well you are doing. Submitting your solution to Lisam is only necessary if you do not submit to CodaBench by yourself.

Submission and evaluation

1. Double-check if your predictions.txt file is correctly formatted as per the instructions.
2. Zip predictions.txt as predictions.zip.
3. Navigate to “My Submissions” in our competition page in CodaBench. Upload predictions.zip (not .txt!) here and wait for your result; see the Figure below.

- After the “Status” has become “Finished”, you will see your accuracy result in “Score” (sometimes this appears empty, and you need to refresh the webpage). Then, click “Add to leaderboard” to send your results to the leaderboard

The screenshot shows the CodaBench submission interface. At the top, there's a 'Final' tab. Below it, two counters show submission limits: 'Number of submissions used for the day' (1 out of 2) and 'Number of total submissions used' (1 out of 10). The 'Submission upload' section includes a 'Submit as:' dropdown set to 'Yourself' and a file upload icon. Below this, a red arrow points to a submission entry 'Running predictions.zip (ID - [redacted])' with the text 'Detailed results (e.g., confusion matrix) are shown here.' Below the submission list, there's a search bar and a 'Status' dropdown. A table lists submissions with columns: ID #, File name, Date, Status, Score, and Actions. The first row shows a submission with ID [redacted], file name 'predictions.zip', date '2025-04-04 13:44', status 'Finished', and an empty score. A red arrow points to the 'Actions' column for this submission, with the text 'Click this to send your results to the leaderboard'.

Final

Number of submissions used for the day
1 out of 2

Number of total submissions used
1 out of 10

Submission upload

Submit as: ?
Yourself

Running predictions.zip (ID - [redacted])

Detailed results (e.g., confusion matrix) are shown here.

Search... Status

ID #	File name	Date	Status	Score	Actions
[redacted]	predictions.zip	2025-04-04 13:44	Finished		

Click this to send your results to the leaderboard

Rules and remarks of the competition

- You will only be able to submit your predictions during given time intervals and the number of submissions that you can make will be limited. The details (and possibly additional rules) will be stated on the CodaBench website.
- You are limited to a maximum of 1 submission per day and 17 submissions in total during the competition.
- It is very important that you precisely follow the required construction (e.g., filename and formatting) of the submission file predictions.txt and predictions.zip. Otherwise, your results will not be computed.
- The metric “accuracy” alone does not given the complete picture of how well your model is performing, in particular when there are multiple classes. Although the leaderboard is based on accuracy, your report should contain a comprehensive analysis of your classifier (e.g., with a confusion matrix and F1 score).

9. Project seminar

In addition to the written report, the project will be examined during a project seminar (see *Section 9: Important Dates*). The seminar will be split into two parts. First, the “winners” (and possibly runners-up) in the leaderboard competition will get the opportunity to present your solution(s). Second, we will have discussions in smaller groups to share our experiences from the project with each other. Discussion questions will be handed out during the seminar.

Participation in the project seminar is mandatory.

If you are unable to participate, we will ask you to submit an individual reflection document instead.

10. Important Dates

2025-04-11	Group signup	Sign up to a group in Lisam, 5 students per group.
2025-04-28	Leaderboard opens	
2025-05-14	Leaderboard closes	Also submission deadline if you prefer to submit to the leaderboard through Lisam (only required for groups that do not have access to CodaBench)
2025-05-20	Submission of report	Submit through Lisam
2025-05-22	Project seminar	